```
import pandas as pd

# Path to the CSV file
csv_path = '/content/spam.csv'

# Read the CSV file
df = pd.read_csv(csv_path, encoding='latin-1')

# Display the first few rows
print(df.head())
```

```
     v1                                                 v2 Unnamed: 2
\
0   ham  Go until jurong point, crazy.. Available only ...         NaN

1   ham                      Ok lar... Joking wif u oni...         NaN

2  spam  Free entry in 2 a wkly comp to win FA Cup fina...         NaN

3   ham  U dun say so early hor... U c already then say...         NaN

4   ham  Nah I don't think he goes to usf, he lives aro...         NaN


  Unnamed: 3 Unnamed: 4
0        NaN        NaN
1        NaN        NaN
2        NaN        NaN
3        NaN        NaN
4        NaN        NaN
```

Perform some preprocessing operations..

```
#delete unnecesary columns...
df.drop(df.columns[[2, 3,4]],axis=1,inplace=True)

# Display the first few rows after deleting columns
print(df.head())
```

```
     v1                                                 v2
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                      Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...
```

```
#print whole dataset...
print(df)
```

```
        v1                                                 v2
0      ham  Go until jurong point, crazy.. Available only ...
```

```
1      ham                      Ok lar... Joking wif u oni...
2     spam  Free entry in 2 a wkly comp to win FA Cup fina...
3      ham  U dun say so early hor... U c already then say...
4      ham  Nah I don't think he goes to usf, he lives aro...
...    ...                                                ...
5567  spam  This is the 2nd time we have tried 2 contact u...
5568   ham              Will Ì_ b going to esplanade fr home?
5569   ham  Pity, * was in mood for that. So...any other s...
5570   ham  The guy did some bitching but I acted like i'd...
5571   ham                         Rofl. Its true to its name

[5572 rows x 2 columns]
```

```
#Count the number of spam and ham msg...
df['v1'].value_counts()
```

```
v1
ham     4825
spam     747
Name: count, dtype: int64
```

```
df.isnull().sum()
```

```
v1    0
v2    0
dtype: int64
```

```
#delete duplicate data...
df.drop_duplicates(inplace=True)
```

```
#print whole dataset after deleting duplicate data...
print(df)
```

```
        v1                                                 v2
0      ham  Go until jurong point, crazy.. Available only ...
1      ham                      Ok lar... Joking wif u oni...
2     spam  Free entry in 2 a wkly comp to win FA Cup fina...
3      ham  U dun say so early hor... U c already then say...
4      ham  Nah I don't think he goes to usf, he lives aro...
...    ...                                                ...
5567  spam  This is the 2nd time we have tried 2 contact u...
5568   ham              Will Ì_ b going to esplanade fr home?
5569   ham  Pity, * was in mood for that. So...any other s...
5570   ham  The guy did some bitching but I acted like i'd...
5571   ham                         Rofl. Its true to its name

[5169 rows x 2 columns]
```

```
#Count the number of spam and ham msg after deleting duplicate data...
df['v1'].value_counts()
```

```
v1
ham     4516
spam     653
Name: count, dtype: int64
```

```
# Separate the data into features and class or label
X = df['v2']  # The message column is named 'v2'
y = df['v1']  # The label column is named 'v1'

# Convert labels or class to binary format
y = y.map({'ham': 0, 'spam': 1})
```

Spliting the dataset into Train and Test data..

```
# Split the data into training and testing sets

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Convert text data to TF-IDF **features**

```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(max_features=3000)
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
```

Training and Testing the Naive Bayes **classifiers**

```
# Import necessary packages and library...
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Initialize the classifier
classifier = MultinomialNB()

# Train the classifier
classifier.fit(X_train_tfidf, y_train)

# Make predictions
y_pred = classifier.predict(X_test_tfidf)

# Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test,
y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.97678916827853
Classification Report:
              precision    recall  f1-score   support

           0       0.97      1.00      0.99       889
           1       1.00      0.83      0.91       145

    accuracy                           0.98      1034
   macro avg       0.99      0.92      0.95      1034
weighted avg       0.98      0.98      0.98      1034

Confusion Matrix:
 [[889   0]
 [ 24 121]]
```

Training and Testing the LogisticRegression **classifiers**

```
# Import necessary packages and library...
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Initialize the classifier
classifier = LogisticRegression()

# Train the classifier
classifier.fit(X_train_tfidf, y_train)

# Make predictions
y_pred = classifier.predict(X_test_tfidf)

# Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test,
y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.971953578336557
Classification Report:
              precision    recall  f1-score   support

           0       0.97      1.00      0.98       889
           1       0.97      0.83      0.89       145

    accuracy                           0.97      1034
   macro avg       0.97      0.91      0.94      1034
weighted avg       0.97      0.97      0.97      1034

Confusion Matrix:
```

```
 [[885   4]
 [ 25 120]]
```

Training and Testing the Support Vector Machines **classifier**

```
# Import necessary packages and library...
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Initialize the classifier
classifier = SVC(kernel='linear', probability=True)  # Using a linear
kernel

# Train the classifier
classifier.fit(X_train_tfidf, y_train)

# Make predictions
y_pred = classifier.predict(X_test_tfidf)

# Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test,
y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.9874274661508704
Classification Report:
              precision    recall  f1-score   support

           0       0.99      1.00      0.99       889
           1       0.98      0.93      0.95       145

    accuracy                           0.99      1034
   macro avg       0.98      0.96      0.97      1034
weighted avg       0.99      0.99      0.99      1034

Confusion Matrix:
 [[886   3]
 [ 10 135]]
```